

An Image is Worth 16×16 Words

Transformers for Image Recognition at Scale (Dosovitskiy et al., ICLR 2021)

Seung Ho Go

August 30, 2026

Table of contents

- 1 Where ViT sits
- 2 Architecture
- 3 What changed from the vanilla Transformer
- 4 Experiments: scale decides
- 5 Inspecting what ViT learned

The claim, in one sentence

Treat an image as a **sequence of patches**, and a standard Transformer encoder works **almost unmodified** — provided you **buy with data** what a CNN gets for free.

NLP, by 2020

Transformer is the default. Pre-train on a large corpus, fine-tune on a small task. Past 100B parameters, **still no sign of saturation**.

Vision, by 2020

CNNs still dominate. Attention gets *bolted onto* CNNs, or replaces parts of them, usually with **specialised attention patterns** that do not accelerate well on modern hardware.

*The question: drop the CNN entirely and apply a **standard** Transformer. What happens?*

What is actually new here

Read only the abstract and you would think “Transformers applied to images, for the first time”. That is **not** the contribution.

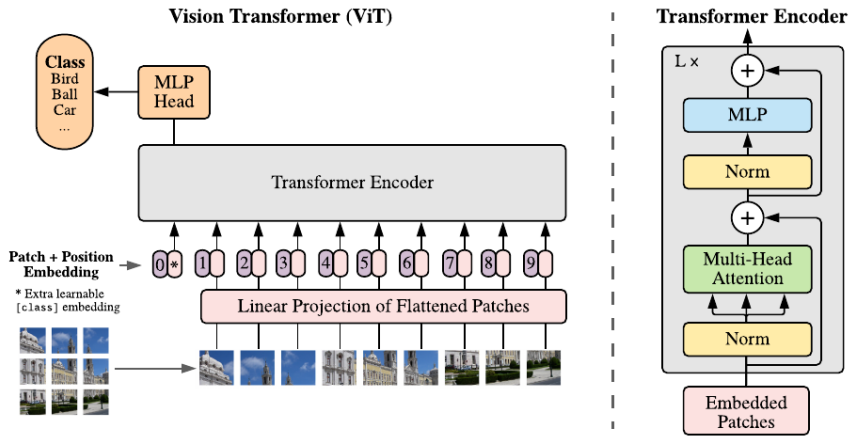
Closest prior work — Cordonnier et al. (2020)

Already extracted 2×2 patches and ran **full self-attention** on top. Structurally very close to ViT. But: patches that small confine the model to **low-resolution** images, and it was **not pre-trained at scale**.

The real delta: not the architecture, but the **evidence** — that a *vanilla* Transformer beats state-of-the-art CNNs **once pre-training is large enough**, demonstrated at **medium resolution** and **300M images**.

Keep this framing for the rest of the talk: the interesting question is not *why was it designed this way*, but **why did it work**.

Overview



Left: the full model. Right: one encoder block.

Only the **encoder** of the 2017 Transformer is used — the output is a single class, not a sentence, so there is nothing for a decoder to do.

Step 1 — turning an image into a token sequence

A Transformer only accepts `B, seq_len, D`. The image must be reshaped into that.

`B, 3, 224, 224` $\longrightarrow$ split into $P = 16$ patches $\longrightarrow$ `B, 196, 768`

- $N = HW/P^2 = (224 \times 224)/16^2 = \mathbf{196}$ patches
- flattened patch length $P^2 \cdot C = 16^2 \times 3 = \mathbf{768}$
- this N is the sequence length — the analogue of the number of words in a sentence

vanilla: a word arrives as a one-hot index and `nn.Embedding` **picks a row**.

ViT: a patch arrives as **continuous pixel values**, so a lookup is impossible — a **linear map** is needed instead.

In code this is one `nn.Conv2d(3, D, kernel_size=16, stride=16)`, which fuses this step and the next. Mathematically identical.

Step 2 — “Linear Projection of Flattened Patches”

Linear Projection of Flattened Patches and *patch embedding* are **two names for the same thing**.

$$\boxed{B, 196, 768} \longrightarrow \mathbf{E} \in \mathbb{R}^{768 \times D} \longrightarrow \boxed{B, 196, D}$$

- **E** is **one learned matrix**, and **all 196 patches pass through the same one** — weight sharing across positions.
- Because of that sharing, **E** survives a change in N . **This matters later** (fine-tuning at higher resolution).

Easy to misread

ViT-B/16 happens to have $P^2C = 768$ and $D = 768$, so the projection looks like an identity. It is a coincidence — it is still a learned linear map, and ViT-L/16 makes it obvious: $768 \rightarrow 1024$.

Step 3 — the class token and position embeddings

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}$$

- The brackets are a **concatenation**; the $+$ is a genuine **element-wise sum**. *Two different operations in one line.*
- $\mathbf{x}_{\text{class}}$ — BERT's [class] token. Already D -dimensional and **learned**, so it does *not* go through $\mathbf{E}$.
- $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ — 197 rows, the class token included.

$$(\text{B}, 196, D) \rightarrow \text{prepend CLS} \rightarrow (\text{B}, 197, D) \rightarrow + \mathbf{E}_{\text{pos}} \rightarrow (\text{B}, 197, D)$$

vanilla used a fixed sinusoidal *encoding*; **ViT** uses a **learned 1D position embedding** — a parameter, not a formula.

Why a class token at all?

Classification needs **one fixed-size vector**, but the encoder returns 197 of them.

Averaging (GAP)

Weights every patch identically at $1/196$. The **selection** that attention is good at is thrown away at the last step.

Class token

Starts as an **empty slot** and collects information from the patches through self-attention, with **learned** weights. *What* to summarise becomes learnable too.

Do not confuse it with the decoder's <sos>: <sos> starts autoregressive generation *under masking*; CLS sees every patch and produces *one* summary. Different jobs.

Initialised to the zero vector. The paper also compares it against GAP — see backup.

Step 4 — the encoder

$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell}, \quad \ell = 1 \dots L$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

$\mathbf{z}_L^0$ is **row 0** of the last layer — the class token. The other 196 rows are **discarded** for classification.

	Layers	D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Notation: ViT-L/16 = Large with 16×16 patches. Sequence length scales with $1/P^2$, so /16 is far more expensive than /32.

Inductive bias — the conceptual centre of the paper

A CNN bakes three things into every layer

- **locality** — nearby pixels first
- **2D neighbourhood structure** — it knows what borders what
- **translation equivariance**

ViT injects 2D structure in exactly two places

- ① cutting the image into patches
- ② adjusting $\mathbf{E}_{\text{pos}}$ when the fine-tuning resolution changes

Everywhere else self-attention is **global** from layer 1, and $\mathbf{E}_{\text{pos}}$ at initialisation knows **nothing** about 2D position.

The prediction this sets up

Having given up what a CNN gets for free, ViT **should lose when data is scarce**. Whether that prediction holds is what the entire experimental section is about.

Six deliberate deviations

	Transformer (2017)	ViT (2021)
Parts used	encoder + decoder	encoder only
LayerNorm	Post-LN, $\text{LN}(x + F(x))$	Pre-LN , $x + F(\text{LN}(x))$
MLP nonlinearity	ReLU	GELU
Position info	sinusoidal encoding (fixed)	learned 1D embedding
Pooling	—	class token
Input embedding	one-hot $\rightarrow$ row lookup	flatten $\rightarrow$ linear map
Classifier head	—	MLP (pre-train) / 1 linear layer (fine-tune)

*Pre-LN + GELU is not a ViT invention — GPT-2 already used it. That is consistent with the paper's stated goal of **minimal modification**.*

Why Pre-LN

The difference shows up in the gradient that reaches the previous block.

Post-LN — the identity path gets **multiplied** by the LayerNorm Jacobian:

$$\frac{\partial \mathbf{z}_\ell}{\partial \mathbf{z}_{\ell-1}} = J_{\text{LN}} \left(I + \frac{\partial F}{\partial \mathbf{z}} \right)$$

Pre-LN — the identity path survives **untouched**:

$$\frac{\partial \mathbf{z}_\ell}{\partial \mathbf{z}_{\ell-1}} = I + \delta_\ell$$

Across L layers

$\prod_\ell (I + \delta_\ell)$ cannot vanish even if every δ_ℓ is zero, but $\prod_\ell J_{\text{LN}}(I + \delta_\ell)$ loses that guarantee — J_{LN} is applied **once per layer**.

The 2017 encoder was **6 layers** and got away with it. **ViT-Huge is 32**.

The cost: the residual stream is never renormalised, so one final LN is needed — that is exactly the $\text{LN}(\mathbf{z}_L^0)$ in Eq. (4).

Against the state of the art

	ViT-H/14 (JFT)	ViT-L/16 (JFT)	ViT-L/16 (I21k)	BiT-L (R152x4)	Noisy Student (EffNet-L2)
ImageNet	88.55	87.76	85.30	87.54	88.4 / 88.5
CIFAR-100	94.55	93.90	93.25	93.51	—
VTAB (19)	77.63	76.28	72.72	76.29	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

The **last row** is the real result. ViT-L/16 beats BiT-L on every task at roughly **1/15 the pre-training cost**. The win is in the **accuracy/compute trade-off**, not in accuracy alone.

Caveat: this comparison is **not controlled** — optimiser, schedule and weight decay all differ. The paper says so, and runs a controlled study separately (§4.4).

Dataset size flips the conclusion

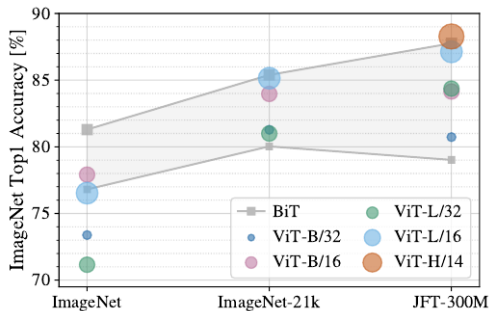


Fig. 3 — ImageNet accuracy vs. pre-training dataset

- **ImageNet (1.3M):** ViT **loses** to BiT ResNets (shaded band). ViT-Large is even **worse** than ViT-Base.
- **ImageNet-21k (14M):** roughly level.
- **JFT-300M:** ViT overtakes — and **the larger the model, the bigger the gain.**

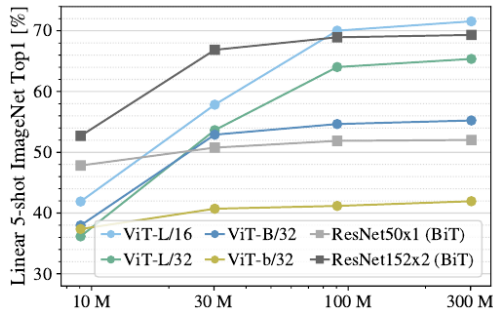


Fig. 4 — few-shot accuracy vs. JFT subset size

Exactly the prediction from the inductive-bias slide. In the paper's phrase, **large scale training trumps inductive bias.**

Hybrids: what if we hand the CNN bias back?

A **hybrid** feeds a **ResNet feature map** to ViT instead of raw patches, with patch size $P = 1$:

$(B, 3, 224, 224) \rightarrow \text{ResNet} \rightarrow (B, C', H', W') \rightarrow \text{flatten} \rightarrow (B, H'W', C') \rightarrow E \rightarrow (B, H'W', D)$

model (JFT)	ImageNet	exaFLOPs
ViT-B/32	80.73	55
R50+ViT-B/32	84.90	106
ViT-L/16 (14ep)	87.12	1567
R50+ViT-L/16 (14ep)	87.12	1668

Small model: **+4.2 points**.

Largest model: **identical to two decimals**, for *more* compute.

At sufficient scale, handing the model an inductive bias **buys nothing** — training already supplies it.

Attention distance: ViT rediscovers early conv layers

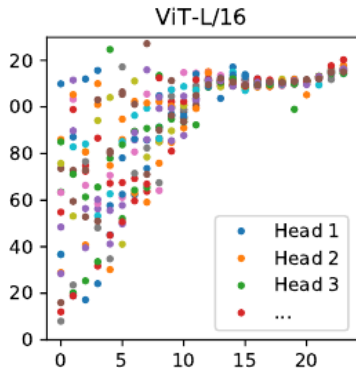


Fig. 7 (right) — ViT-L/16

The analogue of **receptive field size**: mean pixel distance between query and key, weighted by the attention weights.

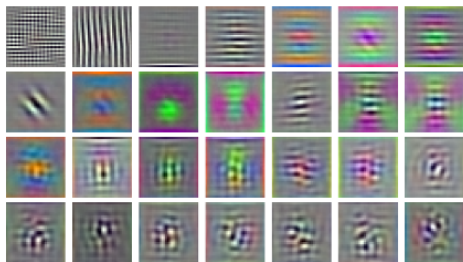
- **Low layers**: heads disagree strongly. Some already span the whole image; others stay in a **very small neighbourhood**.
- **Deeper**: every head attends broadly.

Those narrow low-layer heads are ViT **learning what a CNN's first conv layers do**, from scratch.

Supporting evidence: in the **hybrid**, those localised heads are much less pronounced — the ResNet has already done that job.

The learned patch-embedding filters

RGB embedding filters
(first 28 principal components)



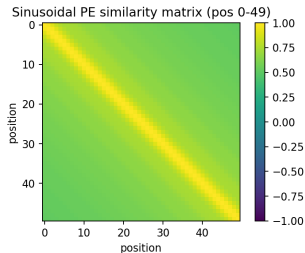
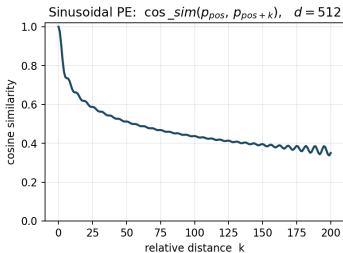
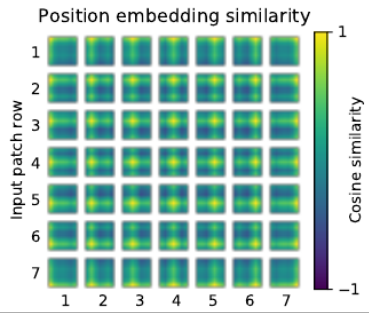
Each **column** of **E** is one conv filter. Treat the D filters as D data points, run PCA, then reshape each principal component back to $P \times P \times 3$ and plot it.

Locality was never given to the model, yet the components look like **edges, gratings and colour-opponent blobs** — the same basis a CNN's first layer converges to.

Fig. 7 (left) — top 28 principal components of **E**, ViT-L/32

Careful with the wording: PCA needs *many* samples. This is PCA **over the set of D filters**, not “PCA per filter”.

Position embeddings rediscover 2D topology



computed directly — sinusoidal PE, $d = 512$

Fig. 7 (center) — learned E_{pos} , ViT-L/32

- **Left (learned):** nearby patches are more similar, and patches sharing a **row or column** light up together.
- **Right (formula):** $\cos_sim = \frac{2}{d} \sum_i \cos(k w_i)$ — decay with distance is **built in by construction**.

Same property, opposite origin. **vanilla** was **told**; **ViT** was given a bare 1D index and **worked it out** from the classification loss alone.

So does telling it about 2D help?

Position embedding	Default	Every layer	Every layer, shared
none	0.61382	—	—
1-D (raster order; the default)	0.64206	0.63964	0.64292
2-D (separate X , Y of size $D/2$, concatenated)	0.64001	0.64046	0.64022
relative (offset $p_q - p_k$ as a logit bias)	0.64032	—	—

Table 8 — ViT-B/16, ImageNet 5-shot linear

- **none** → **any**: about **2.8 points**. Position information itself clearly matters.
- **1D vs. 2D vs. relative**: third decimal place. **Effectively no difference.**

The encoder operates at **patch level** (14×14), not pixel level (224×224). Spatial relations at that resolution are equally easy to learn whichever scheme you use — and, per the previous slide, **it learns them anyway**.

The one place 2D structure must be injected by hand

Fine-tuning at **higher resolution** (384) than pre-training (224) helps. The patch size stays fixed, so the sequence gets longer:

`E_pos: 197 x D` $\longrightarrow$ needed $\longrightarrow$ `E_pos: 577 x D`

Only 197 rows were ever learned. The fix:

drop CLS row $\rightarrow$ `196, D` $\rightarrow$ reshape $\rightarrow$ `14, 14, D` $\rightarrow$ bilinear $\rightarrow$ `24, 24, D` $\rightarrow$ flatten, re-attach CLS

Why this is legitimate

It only works because the **previous slide** established that nearby positions really do have similar embeddings. Interpolating between unrelated random vectors would produce nothing meaningful.

This and patch extraction are the **only two** places 2D bias is hand-injected — and it is the one real weakness of learned PE relative to sinusoidal.

Takeaways

One question ran through the whole talk: **how much 2D inductive bias does ViT actually need?**

Evidence	Answer
Fig. 3, 4	with little data, a lot — it loses to CNNs
Table 6 (hybrids)	with enough data, giving it back buys nothing
Fig. 7 (center)	it learns 2D topology on its own
Table 8	so how you encode position barely matters
§3.2 interpolation	except when resolution changes — then, by hand

What stays unresolved

JFT-300M is proprietary — the headline result cannot be reproduced externally. Table 2 is **not a controlled comparison**. Self-supervised pre-training (§4.6) is still **4 points behind** supervised.

Backup — what actually gets updated

Updated **every mini-batch** (not once per epoch):

- patch embedding $\mathbf{E}$, class token $\mathbf{x}_{\text{class}}$, position embedding $\mathbf{E}_{\text{pos}}$
- per layer, per head: W_Q, W_K, W_V , plus the output projection W_O
- per layer: two sets of LayerNorm γ, β
- per layer: MLP weights W_1, b_1, W_2, b_2
- the final LN and the MLP head

A **sinusoidal** PE would be the one thing *not* updated. ViT's is learned, so it is.

pre-train: Adam ($\beta_1=0.9, \beta_2=0.999$), batch 4096, weight decay 0.1

fine-tune: SGD momentum 0.9, batch 512, no weight decay, grad clipping at norm 1

Backup — how W_Q , W_K , W_V differentiate

One scalar loss, so why do three matrices take on different roles? Because they sit at **different places in the computational graph**, their gradients are **different expressions**:

$$\frac{\partial L}{\partial W_V} \propto \sum_{i,j} \alpha_{ij} x_j^\top (\cdot) \quad (\text{never passes through the softmax derivative})$$

$$\frac{\partial L}{\partial W_Q} \propto \sum_{i,j} g_{ij} x_i^\top K_j, \quad \frac{\partial L}{\partial W_K} \propto \sum_{i,j} g_{ij} x_j^\top Q_i$$

Measured on a small example, the cosine similarity between these gradients is **0.02** for W_Q – W_K and **0.34** for W_Q – W_V — nearly unrelated directions.

Backup — adding two vectors does not destroy either

If $\mathbf{x} = \mathbf{e}_{\text{patch}} + \mathbf{p}$, is the information not lost?

For scalars, yes: $100 = 99+1 = 98+2$, indistinguishable. In D dimensions, no.

- Linear maps distribute: $(\mathbf{a} + \mathbf{b})W = \mathbf{a}W + \mathbf{b}W$. So the attention score splits automatically into **four terms**: content–content, content–position, position–content, position–position.
- In high dimensions two random vectors have cosine similarity $\approx 1/\sqrt{D}$. For $D = 768$ that is about **0.036** — essentially orthogonal.

The right description is not “it recovers the parts” but **“the parts occupy nearly orthogonal directions, so they barely interfere”**. Nothing needs to be stored separately.

Backup — class token vs. GAP, and a lesson

From Appendix D.3:

- 1 The first attempt at global average pooling + a linear classifier **performed very badly**.
- 2 On investigation, it was **neither** the extra token **nor** the pooling.
- 3 **Lowering the learning rate alone** brought GAP level with, or above, the class token (Fig. 9).

The transferable lesson

A conclusion of the form “architecture A is worse than B” turned out to be “A was run with **B’s hyperparameters**”.
Worth suspecting in every architecture-comparison paper.

The paper does **not** explain *why* GAP wants a smaller learning rate. Plausible stories exist; none is verified.

Backup — the learned pattern depends on hyperparameters

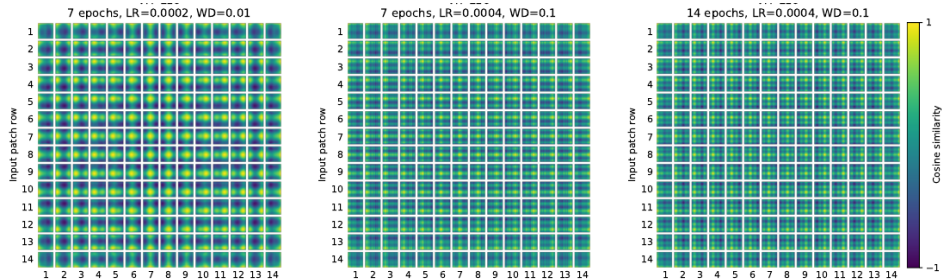


Fig. 10 — same ViT-L/16, different learning rate and weight decay

The clean grid structure of Fig. 7 (center) is **not guaranteed**. It varies visibly with the training hyperparameters.

So “the model learns 2D topology” is an **empirical observation**, not a property. And the blurrier that structure, the **riskier** the interpolation trick at fine-tuning time.

Thank you

Questions?